DOUBLY LINKED LIST

1.Insertion at beginning

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};
struct Node* insertBegin(struct Node *head,int data)
{
    struct Node *newnode = (struct Node*)malloc(sizeof(struct Node
        ));
    newnode->data = data;
    newnode->prev = NULL;
    newnode->next = head;
    if(head != NULL)
        head->prev = newnode;
    head = newnode;
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ",temp->data);
```

```
10 20 30

=== Code Execution Successful ===
```

main.c Share Run Output Clear

```c
    newnode->data = data;
    newnode->prev = NULL;
    newnode->next = head;
    if(head != NULL)
        head->prev = newnode;
    head = newnode;
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *head = NULL;
    head = insertBegin(head,30);
    head = insertBegin(head,20);
    head = insertBegin(head,10);
    display(head);
    return 0;
}
```

```
10 20 30

=== Code Execution Successful ===
```

2.insertion at given position

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};
struct Node* insertPos(struct Node *head,int data,int pos)
{
    struct Node *newnode,*temp;
    int i;
    newnode = (struct Node*)malloc(sizeof(struct Node));
    newnode->data = data;
    if(pos == 1)
    {
        newnode->prev = NULL;
        newnode->next = head;
        if(head != NULL)
            head->prev = newnode;
        return newnode;
    }
    temp = head;
    for(i=1;i<pos-1;i++)
        temp = temp->next;
    newnode->next = temp->next;
```

```
10 20 25 30

=== Code Execution Successful ===
```

```c
    newnode->next = temp->next;
    newnode->prev = temp;
    if(temp->next != NULL)
        temp->next->prev = newnode;
    temp->next = newnode;
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *head = NULL;
    head = insertPos(head,10,1);
    head = insertPos(head,20,2);
    head = insertPos(head,30,3);
    head = insertPos(head,25,3);
    display(head);
    return 0;
}
```

```
10 20 25 30

=== Code Execution Successful ===
```

3.Insertion at end

Programiz
C Online Compiler

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};
struct Node* insertEnd(struct Node *head, int data)
{
    struct Node *newnode = (struct Node*)malloc(sizeof(struct Node
        ));
    newnode->data = data;
    newnode->next = NULL;
    if(head == NULL)
    {
        newnode->prev = NULL;
        return newnode;
    }
    struct Node *temp = head;
    while(temp->next != NULL)
        temp = temp->next;
    temp->next = newnode;
    newnode->prev = temp;
    return head;
}
```

Share Run

Output

```
10 20 30

=== Code Execution Successful ===
```

main.c

```c
    }
    struct Node *temp = head;
    while(temp->next != NULL)
        temp = temp->next;
    temp->next = newnode;
    newnode->prev = temp;
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *head = NULL;
    head = insertEnd(head,10);
    head = insertEnd(head,20);
    head = insertEnd(head,30);
    display(head);
  return 0;
}
```

Share Run

Output

```
10 20 30

=== Code Execution Successful ===
```

4.deletion at beginning

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};
struct Node* deleteBegin(struct Node *head)
{
    if(head == NULL)
    {
        printf("List is empty\n");
        return NULL;
    }
    struct Node *temp = head;
    head = head->next;
    if(head != NULL)
        head->prev = NULL;
    free(temp);
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
```

Output

```
Before deletion:
10 20 30
After deletion:
20 30

=== Code Execution Successful ===
```

Programiz C Online Compiler

```c
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = (struct Node*)malloc(sizeof(struct Node));
    n2 = (struct Node*)malloc(sizeof(struct Node));
    n3 = (struct Node*)malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->next = n2;
    n1->prev = NULL;
    n2->next = n3;
    n2->prev = n1;
    n3->next = NULL;
    n3->prev = n2;
    printf("Before deletion:\n");
    display(head);
```

Output

```
Before deletion:
10 20 30
After deletion:
20 30

=== Code Execution Successful ===
```

```c
    }
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = (struct Node*)malloc(sizeof(struct Node));
    n2 = (struct Node*)malloc(sizeof(struct Node));
    n3 = (struct Node*)malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->next = n2;
    n1->prev = NULL;
    n2->next = n3;
    n2->prev = n1;
    n3->next = NULL;
    n3->prev = n2;
    printf("Before deletion:\n");
    display(head);
    head = deleteBegin(head);
    printf("\nAfter deletion:\n");
    display(head);
    return 0;
}
```

```
Before deletion:
10 20 30
After deletion:
20 30

=== Code Execution Successful ===
```

5.deletion at end

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};
struct Node* deleteEnd(struct Node *head)
{
    if(head == NULL)
    {
        printf("List is empty\n");
        return NULL;
    }
    struct Node *temp = head;
    while(temp->next != NULL)
        temp = temp->next;
    if(temp->prev != NULL)
        temp->prev->next = NULL;
    else
        head = NULL;
    free(temp);
    return head;
}
void display(struct Node *head)
```

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

main.c

```c
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = malloc(sizeof(struct Node));
    n2 = malloc(sizeof(struct Node));
    n3 = malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->prev = NULL;
    n1->next = n2;
    n2->prev = n1;
    n2->next = n3;
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

main.c

```c
    }
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = malloc(sizeof(struct Node));
    n2 = malloc(sizeof(struct Node));
    n3 = malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->prev = NULL;
    n1->next = n2;
    n2->prev = n1;
    n2->next = n3;
    n3->prev = n2;
    n3->next = NULL;
    printf("Before deletion:\n");
    display(head);
    head = deleteEnd(head);
    printf("\nAfter deletion:\n");
    display(head);
    return 0;
}
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

6.deletion at given position

C Online Compiler

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};
struct Node* deletePos(struct Node *head,int pos)
{
    struct Node *temp = head;
    int i;
    if(pos == 1)
    {
        head = temp->next;
        if(head != NULL)
            head->prev = NULL;
        free(temp);
        return head;
    }
    for(i=1;i<pos;i++)
       temp = temp->next;
    if(temp->next != NULL)
        temp->next->prev = temp->prev;
    temp->prev->next = temp->next;
    free(temp);
```

```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successful ===
```

Programiz

main.c

C Online Compiler

Share

Run

Output

```c
    free(temp);
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = malloc(sizeof(struct Node));
    n2 = malloc(sizeof(struct Node));
    n3 = malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->prev = NULL;
    n1->next = n2;
```

```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successful ===
```

Programiz
C Online Compiler

main.c

Share

Run

```c
    }
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = malloc(sizeof(struct Node));
    n2 = malloc(sizeof(struct Node));
    n3 = malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->prev = NULL;
    n1->next = n2;
    n2->prev = n1;
    n2->next = n3;
    n3->prev = n2;
    n3->next = NULL;
    printf("Before deletion:\n");
    display(head);
    head = deletePos(head,2);
    printf("\nAfter deletion:\n");
    display(head);
    return 0;
}
```

Output

```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successful ===
```